

Ultimate Android Developer Guide — 2026 Edition

Это не tutorial с нуля, а справочник для собеседований и самопроверки: закрыть пробелы в фундаменте и свериться с актуальным стеком 2026 года. Структура стека, вокруг которого всё построено: **Jetpack Compose + Kotlin Coroutines/Flow + MVI (Actor/Reducer/NewsPublisher) + Dagger 2 (ручной DI) + кастомная Fragment-навигация + Room/DataStore + Retrofit**. XML и RxJava упоминаются только там, где вы реально с ними столкнётесь на практике — при миграции легаси-кода.

Актуальность на момент написания: **Android 16 (API 36)** — текущий обязательный target для Google Play (дедлайн 31 августа 2026), только что вышел **Android 17 (API 37)**.

1. Kotlin: фундамент, который часто объясняют неправильно

`internal` — это не «папка»

`internal` делает член видимым в пределах Kotlin **модуля** — а модуль это единица компиляции: Gradle source set (в androidx-проекте обычно = один Gradle-модуль), IntelliJ module, Maven-проект. Это не физическая папка и не пакет. Практическое следствие: если положить файлы двух разных Gradle-модулей в одну папку на диске, `internal` их не свяжет — видимость определяется тем, как собран граф модулей, а не файловой структурой.

Контракт `equals()` / `hashCode()`

Формальный контракт: если `a.equals(b) == true`, то `a.hashCode() == b.hashCode()` **обязательно**. Обратное — нет: разные объекты вполне могут иметь одинаковый `hashCode()` (это коллизия, не бар). `hashCode()` по умолчанию — это не «адрес в памяти» как таковой, это identity hash, который JVM вычисляет один раз и не пересчитывает, даже если GC физически переместит объект в памяти (это не специфицировано стандартом, детали зависят от JVM-реализации, но опираться на «hashCode = адрес» нельзя).

Для `data class` `equals`/`hashCode`/`copy` генерируются по свойствам **primary constructor**. Свойства, объявленные в теле класса, в сравнение не попадают — частый источник багов при рефакторинге:

```
kotlin
```

```
/** Осторожно: extraFlag не участвует в equals/hashCode/copy */  
data class UserState(val id: String, val name: String) {  
    var extraFlag: Boolean = false  
}
```

`finalize()` — не используйте вообще

`Object.finalize()` deprecated начиная с Java 9 и помечен «for removal» в актуальных версиях JDK. Никогда не полагайтесь на него для освобождения ресурсов — GC может вызвать его поздно или не вызвать вовсе. Правильный путь: `AutoCloseable` + `use { }`, либо явный `close()`/`release()`, вызываемый из жизненного цикла компонента (`onCleared()` во ViewModel, `Dispatcher.close()` и т.п.).

`Executors` создают потоки, а не наоборот

`Executors.newFixedThreadPool(n)` и подобные фабрики **создают и управляют** пулом worker-потоков — это их прямая обязанность. Утверждение «исполнители не создают потоки» — ошибка, обычно результат путаницы с корутинами: у корутин действительно нет создания нового OS-потока на каждый `launch` — миллион корутин ≠ миллион потоков, потому что они мультиплексируются на пул потоков через `Dispatcher`. Но сам пул потоков кто-то должен создать — и это `Executor/Dispatcher`.

Sealed classes/interfaces как основа состояний

`sealed` даёт компилятору исчерпывающий список наследников — `when` без `else` компилируется, только если покрыты все ветки, и это ловит ошибки на этапе компиляции, а не в рантайме. Именно поэтому STATE/EFFECT/ACTION/NEWS в MVI моделируются через `sealed class`/`sealed interface`, а не через `enum` + поле с данными:

kotlin

```
/** State description for UI layer */
sealed class ProductListState {
    /**
     * State with data ready for display.
     * @param items список загруженных товаров
     */
    data class Content(val items: List<Product>) : ProductListState()
    /** Загрузка в процессе */
    data object Loading : ProductListState()
    /** Ошибка загрузки */
    data class Error(val message: String) : ProductListState()
}
```

`sealed interface` вместо `sealed class` выбирают, когда наследнику не нужен свой конструктор/общее состояние — например, для ACTION, где варианты совсем разнородны.

Вопрос на собеседовании: «Почему `sealed class` лучше `enum class` с полем данных?» — джун скажет «так красивее»; мидл скажет «exhaustive when»; синьор добавит, что `sealed` разрешает разным веткам иметь **разный набор полей** (enum — нет, у всех констант одна и та же форма), и что new subclass ломает компиляцию во всех `when` по всему проекту — это сознательный trade-off в пользу safety при рефакторинге.

Scope functions — не украшательство, а семантика

Функция	Контекст	Возврат	Когда
<code>let</code>	<code>it</code>	результат лямбды	трансформация + null-safety (<code>x?.let { }</code>)
<code>run</code>	<code>this</code>	результат лямбды	группировка вычислений, инициализация с обращением к <code>this</code>
<code>apply</code>	<code>this</code>	сам объект	конфигурирование объекта (builder-стиль)
<code>also</code>	<code>it</code>	сам объект	побочный эффект (логирование) без изменения цепочки
<code>with</code>	<code>this</code>	результат лямбды	группировка вызовов на не-nullable объекте

Частая ошибка — использовать `apply` там, где нужен `also`, и терять читаемость: `apply` подразумевает «настрой этот объект», `also` — «сделай что-то попутно, объект не трогаем».

2. Coroutines: структурная конкурентность

`CoroutineScope` — это не «способ запустить асинхронный код», это владение жизненным циклом

Каждый `launch`/`async` привязан к `Job` родительского scope. Отмена родителя **каскадно** отменяет всех детей — это и есть structured concurrency: у асинхронной работы всегда есть владелец, и работа не может «утечь» дольше жизни владельца. Во ViewModel для этого есть `viewModelScope` (отменяется в `onCleared()`), в Compose — `rememberCoroutineScope()`/`LaunchedEffect`.

`Job` vs `SupervisorJob`

С обычным `Job` падение **любого** ребёнка отменяет всех сиблингов и сам `scope`. С `SupervisorJob` падение одного ребёнка не трогает остальных — это то, что должно быть в основе `viewModelScope`-подобных `scope` верхнего уровня: одна упавшая корутина не должна убивать весь экран.

kotlin

```
/** Actor выполняет побочные эффекты и не должен ронять весь scope при ошибке */
class ProductListActor @Inject constructor(
    private val repository: ProductRepository,
) : Actor<ProductListState, ProductListAction, ProductListEffect> {

    override fun invoke(action: ProductListAction, state: ProductListState): ProductListEffect {
        when (action) {
            is ProductListAction.ScreenOpened -> {
                emit(ProductListEffect.Loading)
                val result = repository.fetchProducts() // suspend, кидает наверх
                emit(ProductListEffect.DataLoaded(result))
            }
        }
    }.catch { e ->
        Timber.tag(TAG).e("mylog 001: Failed to load products: ${e.message}")
        emit(ProductListEffect.ErrorOccurred(e.message.orEmpty())) // Actor не должен ронять scope
    }.flowOn(Dispatchers.IO)

    private companion object {
        const val TAG = "ProductListActor"
    }
}
```

Обратите внимание на `.catch` — это буквальная реализация требования гайдлайна «Actor must NOT throw — return EFFECT or emptyFlow()».

Отмена кооперативна

`cancel()` не останавливает корутину принудительно — он выставляет флаг, и корутина обязана сама проверять его (это делают все `suspend`-функции из `kotlinx.coroutines` автоматически на каждой точке приостановки) либо явно вызывать `ensureActive()`/`yield()` в CPU-bound циклах без `suspend`-точек. Долгий `while` без `suspend`-вызовов внутри — классическая причина «отмена не работает», и это конкретный вопрос, который отличает мидла от синьора: мидл знает про `cancel()`, синьор знает, что нужно самому кооперироваться в `hot loop`.

`withContext` vs `async` + `await`

`withContext` — для одной последовательной `suspend`-операции со сменой диспетчера, возвращает результат напрямую и не даёт параллелизма. `async` — когда реально нужен параллелизм: несколько независимых операций, которые можно выполнить одновременно и потом собрать через `awaitAll()`. Частая ошибка мидла — писать `async { }.await()` там, где по смыслу нужен просто `withContext` — работает, но вводит лишний `Deferred` без пользы.

Обработка исключений: `try/catch` vs `CoroutineExceptionHandler`

`CoroutineExceptionHandler` ловит только необработанные исключения из **корневых** корутин (`launch`, не `async`) и не может «поймать» исключение и продолжить выполнение — на момент его вызова корутина уже мертва. Для контролируемой обработки ошибок внутри бизнес-логики используется `try/catch` или Flow-оператор `.catch { }`, а `CoroutineExceptionHandler` — последний рубеж для логирования того, что осталось необработанным (crash-репортинг).

Вопрос на собеседовании: «Что будет, если бросить исключение внутри `async { }` и не вызвать `await()`?» — джун не знает; мидл скажет «исключение потеряется до `await()`»; синьор уточнит: оно не потеряется навсегда — `Deferred` хранит его и перебросит при `await()`, но если `await()` вообще не вызвать, исключение всё равно распространится вверх по иерархии `Job` — просто отложено, и это частый источник «тихих» крашей далеко от места ошибки.

3. Flow, StateFlow, SharedFlow

Cold vs hot — ключевое отличие

`Flow` (обычный, cold) не выполняет код до появления коллектора и выполняет его **заново для каждого** нового коллектора. `StateFlow`/`SharedFlow` (hot) выполняются независимо от наличия подписчиков и делят один и тот же поток данных между всеми коллекторами.

	<code>StateFlow</code>	<code>SharedFlow</code>
Начальное значение	обязательно	нет (если не настроить <code>replay</code>)
Дедупликация	да, по <code>equals()</code> (встроенный <code>distinctUntilChanged</code>)	нет, по умолчанию эмитит всё
Использование	STATE в MVI — всегда есть «текущее» состояние	NEWS/one-shot события

Именно поэтому в гайдлайне `state: StateFlow<STATE>`, а `news: Flow<NEWS>` — по факту `SharedFlow` с буферизацией под капотом («`queued until collected`»), чтобы one-shot-событие не потерялось, если экран был в фоне в момент эмита, но при этом не повторялось при повторной подписке — в отличие от `StateFlow`, который всегда отдаёт последнее значение заново.

`repeatOnLifecycle` — единственно верный способ собирать Flow в UI

kotlin

```
/** Собирает state только пока Fragment хотя бы STARTED — иначе лишняя работа */
viewLifecycleOwner.lifecycleScope.launch {
    viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.state.collect { state -> render(state) }
    }
}
```

Простой `lifecycleScope.launch { flow.collect { } }` без `repeatOnLifecycle` продолжает собирать значения, даже когда экран не виден (`STOPPED`) — лишняя работа и потенциальный краш при обращении к уже уничтоженным View. В Compose эквивалент — `collectAsStateWithLifecycle()`, не голый `collectAsState()`.

Операторы, которые реально нужны на практике

- `flowOn(Dispatchers.IO)` — меняет диспетчер для **upstream** (всего, что выше по цепочке), а не для оператора рядом с ним — частая путаница у мидлов.
- `distinctUntilChanged()` — избежать лишней `recomposition` при одинаковых последовательных значениях.
- `debounce()` / `sample()` — для поискового ввода, `resize`-событий.
- `combine()` vs `zip()` — `combine` пересчитывает результат при любом новом значении в любом источнике; `zip` ждёт пару значений от каждого по очереди. Для UI почти всегда нужен `combine`.

Тестирование Flow

`kotlinx-coroutines-test` (`runTest`, `TestDispatcher`) + библиотека **Turbine** — стандарт для тестов Flow/StateFlow: `flow.test { assertEquals(expected, awaitItem()) }` вместо ручного `toList()` с таймаутами.

Миграция с RxJava (кратко, только для legacy-контекста)

RxJava	Coroutines/Flow
<code>Observable</code> / <code>Flowable</code>	<code>Flow</code>
<code>Single</code> / <code>Maybe</code>	<code>suspend-функция</code> , возвращающая значение (или <code>null</code>)
<code>Subject</code> / <code>BehaviorSubject</code>	<code>MutableStateFlow</code> / <code>MutableSharedFlow</code>
<code>subscribeOn</code> / <code>observeOn</code>	<code>flowOn</code> (upstream) / смена <code>Dispatcher</code> в точке сбора

Главная ловушка при миграции — не механическая замена типов, а привычка к **hot по умолчанию**: `Subject` в Rx-коде часто использовался там, где по факту была нужна семантика `StateFlow` (последнее значение + дедупликация), и наивная замена на `MutableSharedFlow` без `replay = 1` эту семантику теряет.

i Из гайдлайна проекта: MVI-адаптер (Actor/Reducer/NewsPublisher core) может использовать RxJava **внутри себя** как деталь реализации фреймворка — это не повод писать бизнес-логику фичи на Rx, наружу контракт всегда Coroutines/Flow.

4. Jetpack Compose: state, recomposition, производительность

Recomposition — это не «перерисовка всего», это избирательный повторный вызов composable-функций

Compose отслеживает, какие `State`-объекты читались внутри какого composable, и при изменении значения помечает к рекомпозиции только те composable, что реально его читали (skipping для остальных). Отсюда практический вывод: **где** вы читаете state, влияет на то, что перекомпонуется. Чтение `viewModel.state.value.title` прямо в корне большого экрана заставляет рекомпонироваться весь экран при любом изменении `title`; вынос чтения в маленький вложенный composable сужает область рекомпозиции до него одного.

`remember` / `rememberSaveable` / `derivedStateOf`

- `remember { }` — переживает рекомпозицию, **не** переживает пересоздание Activity/Fragment (process death, поворот экрана без `ViewModel`).
- `rememberSaveable { }` — то же самое + сохраняется в `Bundle` (переживает process death), но только для типов, которые можно положить в `Bundle`/`Parcelable` или для которых задан `Saver`.
- `derivedStateOf { }` — для вычисляемого значения, которое зависит от **другого** state, но должно триггерить рекомпозицию **реже**, чем меняется источник (например: «прокрутили ли список ниже первого элемента» — булево значение, вычисляемое из `scrollState`, которое реально меняется гораздо реже, чем сам scroll offset).

Частая ошибка мидла — использовать `remember(key1) { derivedStateOf { ... } }`, когда сама derived-логика не нужна: `derivedStateOf` даёт выигрыш только когда результат меняется **реже** входных данных; если он меняется с той же частотой — это чистый оверхед.

Side-effect API — когда что использовать

API	Когда
<code>LaunchedEffect(key)</code>	запустить suspend-код при входе в композицию / смене <code>key</code> (подписка на Flow, разовый запрос)
<code>DisposableEffect(key)</code>	нужен явный cleanup при выходе из композиции (listener, callback)
<code>SideEffect { }</code>	синхронизировать Compose-state с не-Compose миром при каждой успешной рекомпозиции (без своего жизненного цикла)
<code>rememberCoroutineScope()</code>	запустить корутину в ответ на событие (клик), а не при входе в композицию

Типичная ошибка джуна — вызвать `suspend`-функцию прямо в теле `composable` вместо `LaunchedEffect`; типичная ошибка мидла — использовать `LaunchedEffect(Unit)` там, где логика на самом деле зависит от какого-то параметра экрана, и он забывает подписать это в `key`, из-за чего эффект не перезапускается при смене данных.

Стабильность (`@Stable` / `@Immutable`) — почему это не «просто аннотация»

Компилятор Compose пропускает рекомпозицию `composable`, если все его параметры «стабильны» и не изменились (`structural equality`). Стабильность параметра компилятор определяет сам для большинства типов, но **не может** доказать стабильность для типов из другого модуля без метаданных, для интерфейсов, для `List`/`Map` (они помечаются нестабильными по умолчанию, потому что теоретически мутируемы, даже если вы используете только `listOf(...)`). Отсюда практика:

```
kotlin

/** Помечаем явно: все поля val, компилятору незначет сомневаться */
@Immutable
data class ProductCardUi(
    val title: String,
    val price: String,
    val imageUrl: String,
)
```

`@Immutable` — контракт «объект никогда не изменится после создания» (сильнее).

`@Stable` — контракт «если поля равны по `equals()`, то и публичное поведение равно» (слабее, применим когда объект технически `mutable`, но `notify` об изменениях идёт правильно, например через `State<T>` внутри).

Вопрос на собеседовании: «Почему `List<Product>` в параметрах `composable` — плохая практика с точки зрения производительности?» — джун не знает, в чём проблема («это же просто список»); мидл скажет «`List` нестабилен, из-за него `composable` не скипается»; синьор добавит: конкретно `ImmutableList` (`kotlinx.collections.immutable`) или `@Immutable`-обёртка решает проблему, но также важно понимать, что это влияет только на **skip**-оптимизацию, а не меняет корректность — приложение и без этого будет работать правильно, просто будет чаще перерисовывать то, что не изменилось, и на больших списках/сложных экранах это реально заметно в профайлере.

Modifier — порядок имеет значение

`Modifier` применяется последовательно слева направо/сверху вниз —
`.padding(16.dp).background(Color.Red)` даёт красный фон **внутри** отступа;
`.background(Color.Red).padding(16.dp)` даёт красный фон, растянутый **под** отступ тоже. Это не деталь синтаксиса, а модель «каждый `Modifier` оборачивает предыдущий результат», и непонимание порядка — источник багов с `layout`, которые сложно диагностировать без этого знания.

5. Compose внутри Fragment (актуально для нашей навигационной архитектуры)

Стек проекта: Compose UI **живёт внутри Fragment**, а не заменяет его — экран целиком строится через `ComposeView` в `onCreateView`, а маршрутизация между экранами остаётся на `FragmentManager` (см. раздел 8), а не на `Navigation Compose`.

kotlin

```
class ProductListFragment : Fragment() {

    @Inject
    lateinit var viewModelFactory: ViewModelProvider.Factory

    private val viewModel: ProductListViewModel by viewModels { viewModelFact

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?,
    ): View = ComposeView(requireContext()).apply {
        // ViewCompositionStrategy обязателен – иначе composition не диспозуи
        // синхронно с уничтожением View, а живёт до GC, что держит подписки
        setViewCompositionStrategy(ViewCompositionStrategy.DisposeOnViewTreeL
        setContent {
            AppTheme {
                val state by viewModel.state.collectAsStateWithLifecycle()
                ProductListScreen(state = state, onAction = viewModel::accept
            }
        }
    }

    override fun onAttach(context: Context) {
        (context.applicationContext as AppComponentProvider).appComponent.inj
        super.onAttach(context)
    }
}
```

Ключевая деталь, о которой часто забывают: `setViewCompositionStrategy`. Без явной стратегии `Compose` по умолчанию держит `composition` живой дольше, чем нужно (до сборки мусора), что не является утечкой памяти в строгом смысле, но держит активными подписки на `Flow` дольше жизни экрана — реальный источник лишней работы в фоне на легаси-коде, который переносили с XML без этой строчки.

Легаси: XML/Views

Если встречаете старый экран на `View`/XML при доработке — не переписывайте его целиком «заодно», если не просили; для точечных изменений `ComposeView` можно встроить и в XML-layout (`<androidx.compose.ui.platform.ComposeView>` тегом) для постепенной миграции экрана без полного рефакторинга за один PR.

6. Архитектура: Clean Architecture + MVI (Actor/Reducer/NewsPublisher)

UDF/SSOT — теория, из которой всё следует

Unidirectional Data Flow: события идут в одну сторону (UI → ACTION → бизнес-логика → новое STATE → UI), состояние никогда не мутируется напрямую из UI-слоя. Single Source of Truth: для каждого куска состояния существует ровно один «хозяин» источника правды (обычно — Repository/State holder), а не несколько копий одного и того же в разных местах, которые можно рассинхронизировать. MVI — это конкретная реализация UDF, где каждый шаг явно назван и типизирован:

kotlin

```
/**
 * @param ACTION намерение пользователя – вход в систему
 * @param EFFECT результат работы Actor – внутреннее событие
 * @param STATE текущее состояние экрана
 * @param NEWS одноразовое событие для UI (toast, навигация)
 */
abstract class MviViewModel<ACTION, EFFECT, STATE, NEWS>(
    initialState: STATE,
    private val actor: Actor<STATE, ACTION, EFFECT>,
    private val reducer: Reducer<STATE, EFFECT>,
    private val newsPublisher: NewsPublisher<ACTION, EFFECT, STATE, NEWS>,
) : ViewModel() {

    private val _state = MutableStateFlow(initialState)
    val state: StateFlow<STATE> = _state.asStateFlow()

    private val _news = MutableSharedFlow<NEWS>(extraBufferCapacity = 1)
    val news: Flow<NEWS> = _news.asSharedFlow()

    fun acceptAction(action: ACTION) {
        viewModelScope.launch {
            actor.invoke(action, _state.value).collect { effect ->
                val previousState = _state.value
                _state.value = reducer.invoke(previousState, effect) // <-- ч
                newsPublisher.invoke(action, effect, previousState)?.let { _
            }
        }
    }
}
```

Почему три отдельных компонента, а не один «умный» ViewModel

- **Actor** — единственное место, где живут side-эффекты (сеть, БД, время). Он **не имеет права** менять state напрямую — только эмитить EFFECT.
- **Reducer** — чистая синхронная функция `((STATE, EFFECT) → STATE)`. Раз она чистая, её можно протестировать без моков вообще: подаёшь пару state+effect на вход, сравниваешь результат.
- **NewsPublisher** — отделяет «состояние, которое должно быть отражено на экране постоянно» (STATE) от «событие, которое должно произойти один раз и не должно повториться при пересоздании UI» (NEWS, например показ снейкбара — если бы это было частью STATE, снейкбар показывался бы заново при каждом повороте экрана).

Это прямое следствие разделения ответственности (SRP) — каждый компонент тестируется в изоляции и не может «случайно» взять на себя чужую обязанность, потому что типы это не позволяют.

Почему не MVVM/MVP как основной паттерн

MVVM обычно не разделяет «что делает эффект» и «как это меняет состояние» — вся логика часто оказывается в одном методе ViewModel, что усложняет unit-тестирование по отдельности. MVP жёстко связывает Presenter с конкретным View-интерфейсом (`interface ProductListView { fun showProducts(...) }`), что плохо сочетается с декларативным UI, где вместо набора императивных методов должен быть один STATE. Для конкретно этого проекта также важно: MVI даёт единую точку входа (`acceptAction`) и единую точку выхода (`state`), что упрощает и **тестирование**, и **воспроизведение бага** — весь UI-баг сводится к вопросу «какая последовательность ACTION привела к этому STATE».

Clean Architecture: слои и мэппинг — это trade-off, а не религия

Классические слои: presentation (ViewModel/Compose) → domain (use case, чистый Kotlin, без Android-зависимостей) → data (Repository impl, источники данных). Строгое соблюдение (отдельный DTO → отдельная domain-модель → отдельная UI-модель с мэппингом на каждой границе) даёт независимость слоёв, но добавляет boilerplate. На практике для маленькой фичи с одним источником данных три отдельных модели — это часто просто лишний код без реальной выгоды; граница себя оправдывает там, где domain-модель реально используется несколькими фичами/источниками данных или где долгоживущий контракт стоит защитить от изменений в API/схеме БД.

Вопрос на собеседовании: «У фичи один экран и один источник данных. Нужны ли отдельные DTO/domain/UI модели?» — джун скажет «да, так по Clean Architecture»; мидл скажет «можно объединить DTO и domain, но UI-модель отдельно ради stability в Compose»; синьор явно назовёт это trade-off'ом между защитой от изменений API и скоростью разработки, и предложит решение по контексту: если API нестабилен или им владеет другая команда — граница нужна; если это внутренний, контролируемый нами эндпоинт под конкретный экран — можно сократить слои, добавив разделение позже, когда появится вторая точка использования (YAGNI).

7. Dependency Injection: Dagger 2 (ручной DI)

Component/Module/Scope — минимальный набор понятий

- `@Module` — класс с `@Provides`/`@Binds`-методами: как именно создать зависимость.
- `@Component` — интерфейс, соединяющий модули с точками внедрения (`inject(target)`); Dagger генерирует реализацию во время компиляции (`DaggerAppComponent`), поэтому ошибки графа зависимостей ловятся **на compile-time**, а не в рантайме, как в чисто reflection-based DI.
- `@Scope` (например `@Singleton`, кастомный `@FeatureScope`) — привязывает время жизни объекта к времени жизни компонента, который его создал. Без scope Dagger создаёт новый экземпляр при каждом обращении (если не `@Binds`/`@Provides` без scope на классе, у которого нет `@Inject`-конструктора с зависимостями, которые сами по себе не scoped).

kotlin

```
@Module
abstract class ProductModule {
    @Binds
    abstract fun bindProductRepository(impl: ProductRepositoryImpl): ProductRepository
}

@FeatureScope
@Component(dependencies = [AppComponent::class], modules = [ProductModule::class])
interface ProductComponent {
    fun inject(fragment: ProductListFragment)

    @Component.Factory
    interface Factory {
        fun create(appComponent: AppComponent): ProductComponent
    }
}
```

Ручной инжект: `injector.inject(this)`

kotlin

```
class ProductListFragment : Fragment() {  
    @Inject lateinit var viewModelFactory: ViewModelProvider.Factory  
  
    override fun onAttach(context: Context) {  
        DaggerProductComponent.factory()  
            .create((context.applicationContext as AppComponentProvider).appC  
            .inject(this) // <-- поля с @Inject заполняются здесь, до super.c  
        super.onAttach(context)  
    }  
}
```

Инъект в `onAttach`, а не в `onCreate` — чтобы поля были готовы уже к моменту, когда фреймворк вызовет `onCreate` (некоторые данные из `arguments`/зависимостей могут понадобиться уже там).

Multibinding — когда фича должна «сама заявить о себе»

`@IntoSet`/`@IntoMap` (обычно вместе с `@StringKey`/`@ClassKey`) позволяют модулю каждой фичи регистрировать себя в общей коллекции без изменения кода, который эту коллекцию использует — классический пример: набор `WorkerFactory`, набор feature-flag провайдеров, набор экранов для deeplink-роутера. Без multibinding пришлось бы держать в одном центральном модуле список «всех фич», что нарушает изоляцию feature-модулей и требует трогать общий файл при любом добавлении новой фичи.

Тестирование с Dagger

Ручной DI (в отличие от `@AndroidEntryPoint`) не требует специального test runner'а — для юнит-тестов зависимости просто передаются в конструктор напрямую, минуя Dagger вообще (`ProductListActor(fakeRepository)`); Dagger нужен только там, где реально собирается граф на реальном приложении — то есть в инструментальных/end-to-end тестах, где можно завести отдельный тестовый `@Component` с `@Provides`-заменами на fake-реализации.

Dagger vs Hilt — почему в этом проекте ручной Dagger

Hilt упрощает boilerplate (генерирует часть компонентов и `@AndroidEntryPoint` за вас), но привязывает граф зависимостей к жизненным циклам Android-компонентов «из коробки» (`@ActivityScoped`, `@FragmentScoped`) — что плохо ложится на архитектуру, где навигация не через `Fragment`-транзакции Jetpack Navigation, а через кастомный `FragmentAttacher` с собственными scope-границами (`mainAttacher`/`tabBarAttacher` — см. ниже). Ручной Dagger даёт полный контроль над тем, когда именно создаётся/уничтожается feature-компонент, что здесь важнее, чем экономия boilerplate.

8. Навигация: кастомный `FragmentAttacher`

Compose-экраны живут внутри `Fragment`, а маршрутизацией занимается не `Navigation Compose`/`NavController`, а собственный `FragmentAttacher` поверх `FragmentManager`.

Два слоя навигации в `MainActivity`

- `mainAttacher` (`R.id.fullScreenContainer`) — полноэкранный оверлей, скрывающий TabBar: авторизация, pincode, хард-апдейт, полноэкранные deeplink-экраны.
- `tabBarAttacher` (`R.id.screen`) — контент под нижней навигацией: основной экран, фичи уровня таба.

kotlin

```
// Открыть полноэкранный фичу с тегом
mainAttacher.pushToStack(pinFragment, PincodeApi.PIN_TAG)
// Закрыть конкретную фичу по тегу
mainAttacher.popStack(PincodeApi.PIN_TAG)
// Закрыть верхний экран (обычный "назад")
mainAttacher.popStack()
// Проверка перед навигацией
if (mainAttacher.isEmpty()) { /* нет полноэкранных оверлеев */ }
```

`mainAttacher` сам управляет видимостью `fullScreenContainer` через `OnBackStackChangeListener` — ручное управление видимостью контейнера не нужно и обычно означает, что где-то в коде продублирована логика, которой не место вне `FragmentAttacher`.

`attach()` — всегда 3-параметрический: `attacher.attach(fm, containerId, lifecycleOwner)`. 2-параметрический вариант deprecated.

Точки входа в фичи — через `featureApi.getPresentationEntry(): Fragment`, а не прямое создание `Fragment` в месте вызова: это единственная точка, где вызывающий код знает, что именно откроется, весь остальной граф фичи остаётся инкапсулирован за её публичным API-модулем.

Почему не Navigation Compose

`NavController` управляет back stack'ом на уровне composable-графа внутри одного `NavHost` — здесь же экраны это `Fragment`, а не composable верхнего уровня, и навигационная модель уже завязана на два разных контейнера с разной семантикой (полноэкранный оверлей vs контент под табами), что не укладывается в один линейный `NavHost`. Смешивание двух систем навигации в одном проекте (`FragmentAttacher` для части экранов и `NavController` для другой) — источник багов с back stack и predictive back, поэтому весь проект держится на одном механизме.

Predictive back (Android 13+, ужесточение в Android 16)

Начиная с Android 16 система жёстче проверяет соответствие Predictive Back API — приложения, которые всё ещё переопределяют `onBackPressed()` напрямую вместо декларативной модели через `OnBackPressedDispatcher`, начинают вести себя некорректно с системным жестом «назад». В этом проекте back обрабатывается централизованно в `handleBackNavigation()` — именно туда и должна стекаться логика predictive back для всего приложения разом, а не в отдельные `Fragment.onBackPressed()`-переопределения по фиче.

FORBIDDEN: `Navigation Compose`, `NavController`, прямой `FragmentTransaction`, deprecated-методы (`push()`, `pop()`, `pushWithoutAddingToStack()`).

9. Хранение данных: Room + DataStore

Room + Flow

DAO с `Flow`-возвращаемым типом автоматически переэмитит запрос при изменении затронутых таблиц — не нужен ручной колбэк на изменение БД:

```
kotlin

@Dao
interface ProductDao {
    /** Реактивный запрос — переэмитится при любом изменении таблицы products */
    @Query("SELECT * FROM products WHERE category = :category")
    fun observeByCategory(category: String): Flow<List<ProductEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertAll(products: List<ProductEntity>)
}
```

Миграции — обязательны с версии 2 схемы и выше (`Migration(1, 2) { database -> database.execSQL(...) }`); `fallbackToDestructiveMigration()` допустим только для данных, которые можно безопасно потерять (локальный кэш), никогда — для данных, которые пользователь мог создать сам и нигде больше не хранятся.

Транзакции — `@Transaction` на `suspend`-функции DAO для нескольких связанных операций (например, вставка родительской сущности + связанных дочерних одним атомарным блоком).

DataStore вместо `SharedPreferences`

`SharedPreferences` синхронный API на `main thread` по умолчанию (`commit()` блокирует, `apply()` асинхронный, но без гарантий порядка и без типобезопасности), не имеет транзакционной согласованности при параллельной записи и не даёт `Flow` из коробки. `DataStore` (Preferences или Proto) — полностью асинхронный, на `Flow`, с атомарными транзакционными обновлениями:

kotlin

```
class UserPreferencesRepository @Inject constructor(  
    private val datastore: DataStore<Preferences>,  
) {  
    private val onboardingCompletedKey = booleanPreferencesKey("onboarding_completed")  
  
    val onboardingCompleted: Flow<Boolean> = datastore.data  
        .map { prefs -> prefs[onboardingCompletedKey] ?: false }  
  
    suspend fun setOnboardingCompleted(completed: Boolean) {  
        datastore.edit { prefs -> prefs[onboardingCompletedKey] = completed }  
    }  
}
```

Preferences DataStore — для простых key-value без строгой схемы. Proto DataStore — когда нужна типобезопасная схема (protobuf) и есть версионирование данных со временем; выбор между ними — вопрос того, насколько критична типобезопасность схемы, а не производительности (оба асинхронны и построены на одном и том же механизме).

10. Сеть: Retrofit + OkHttp

Interceptor: application vs network level

`addInterceptor()` (application interceptor) видит запрос/ответ один раз, до кэширования и редиректов — подходит для добавления заголовков (auth-токен), логирования намерения. `addNetworkInterceptor()` видит каждый физический сетевой вызов, включая повторные при редиректах — подходит для низкоуровневой отладки трафика, но не для логики, которая не должна выполняться повторно при редиректе.

kotlin

```
class AuthInterceptor @Inject constructor(
    private val tokenProvider: TokenProvider,
) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = tokenProvider.getToken()
        val request = chain.request().newBuilder()
            .addHeader("Authorization", "Bearer $token")
            .build()
        return chain.proceed(request)
    }
}
```

Retrofit + suspend — без RxJava/Callback

kotlin

```
interface ProductApi {
    @GET("products")
    suspend fun getProducts(@Query("category") category: String): List<Product>
}
```

Retrofit нативно поддерживает `suspend`-функции с версии, где убрали необходимость в `CallAdapter` для корутин — исключения из сети (`HttpException`, `IOException`) прилетают как обычные Kotlin-исключения, их ловит `Actor.catch { }`, а не отдельный error-callback.

Ошибки: `HttpException` vs `IOException`

`HttpException` — сервер ответил, но кодом ошибки (4xx/5xx) — есть тело ответа, можно распарсить структурированную ошибку API. `IOException` — запрос не дошёл до сервера или ответ не пришёл (нет сети, таймаут) — тела ответа нет вообще. Смешивание этих двух случаев в один общий `catch (e: Exception)` без различения — частая причина неверных сообщений об ошибке пользователю («нет интернета» вместо «сервер вернул 404»).

Сериализация: Kotlin Serialization vs Moshi

Kotlin Serialization — компилятор генерирует сериализатор на этапе компиляции (без reflection), быстрее и безопаснее с точки зрения R8/обфускации по умолчанию. Moshi (с codegen через KSP, не reflection-адаптер) — тоже compile-time, но исторически более гибкий для нестандартных форматов JSON (полиморфизм, кастомные адаптеры для legacy API). Для нового кода по умолчанию — Kotlin Serialization, если нет специфичной причины тянуть Moshi (например, уже существующие кастомные адаптеры в проекте).

11. Фоновая работа и Services: актуальная картина

`Service` ≠ фоновый поток

`Service` по умолчанию выполняется в **главном потоке** приложения — это компонент жизненного цикла, а не механизм многопоточности сам по себе. Если внутри `Service` нужна тяжёлая работа — всё равно нужны корутины/потоки, `Service` только даёт этой работе возможность продолжаться, когда нет видимого UI.

Background execution limits — почему Service больше не универсальный инструмент

Начиная с Android 8 (Oreo) система резко ограничила, что может делать background-сервис в фоне (без видимого UI и без foreground-статуса) — фоновые сервисы, запущенные через `startService()`, система останавливает вскоре после ухода приложения в фон. С тех пор для разных сценариев есть разные официально рекомендованные инструменты, и Service «на всё» — устаревшее мышление:

Сценарий	Инструмент
Отложенная/надёжная фоновая задача (синхронизация, аплоад, работа, которая должна выполняться даже после перезапуска устройства)	WorkManager
Задача, привязанная к жизни экрана/процесса, не должна переживать закрытие приложения	<code>viewModelScope</code> / <code>lifecycleScope</code> , отдельный <code>CoroutineScope</code> уровня приложения при необходимости
Пользователь явно видит, что идёт активный процесс (плеер, навигация, звонок)	Foreground Service с обязательным notification
Разовая немедленная фоновая задача без гарантии переживания смерти процесса	обычная корутина в scope приложения

Foreground Service: обязательное уведомление + типы (Android 14+)

Foreground Service обязан показать notification почти сразу после старта — иначе система выбрасывает `ForegroundServiceDidNotStartInTimeException`. С Android 14 (API 34) обязательно указывать `foregroundServiceType` (например `location`, `mediaPlayback`, `dataSync`) — и не любой, а соответствующий декларированному в Manifest через `<service android:foregroundServiceType="...">`; несоответствие типа реальной задаче — повод для отклонения в Google Play с недавних релизов policy. Android 12+ дополнительно ограничивает **запуск** foreground service из фона (background start restrictions) — приложение, не находящееся на переднем плане, зачастую не может стартовать FGS напрямую и должно использовать `WorkManager`/`JobScheduler` с `expedited`-флагом или exemptions вроде user-initiated actions.

`AsyncTask` — исключить из актуального инструментария полностью

`AsyncTask` deprecated с API 30 и не должен фигурировать как рабочий паттерн ни в новом коде, ни в объяснении «как правильно» — единственный правильный контекст упоминания сегодня: «если увидите `AsyncTask` в легаси-коде, замените на `coroutine` в `viewModelScope` (для UI-привязанной задачи) или на `WorkManager` (для гарантированной фоновой)».

Вопрос на собеседовании: «Почему просто не использовать `GlobalScope.launch` вместо `WorkManager` для загрузки данных в фоне?» — джун не увидит проблемы; мидл скажет «`GlobalScope` не отменяется вместе с чем-либо, это утечка по смыслу»; синьор добавит: дело не только в отмене — `GlobalScope`-корутина не переживает смерть процесса вообще (killed process = работа просто исчезла без следа), тогда как `WorkManager` персистентно хранит задачу и **гарантированно** перезапустит её после перезапуска процесса/устройства (constraints позволяют, например, «выполнить, когда появится сеть» даже если это будет через час) — это разный контракт надёжности, а не просто разный API.

12. Permissions: актуальная модель (2026)

Runtime, не install-time

Начиная с Android 6.0 (API 23) `dangerous`-permissions (камера, геолокация, контакты и т.п.) запрашиваются **во время работы приложения**, а не при установке — пользователь видит системный диалог именно в момент, когда приложение вызывает `requestPermissions()`/`ActivityResultContracts.RequestPermission()`, и может отказать без деинсталляции приложения. `normal`-permissions (интернет, вибрация) по-прежнему выдаются автоматически при установке без диалога — это единственный случай, где «install-time» всё ещё корректно, и именно эта путаница — источник самой распространённой ошибки в объяснении permissions.

kotlin

```
private val requestPermissionLauncher = registerForActivityResult(  
    ActivityResultContracts.RequestPermission()  
) { granted ->  
    if (granted) {  
        viewModel.acceptAction(CameraAction.PermissionGranted)  
    } else {  
        viewModel.acceptAction(CameraAction.PermissionDenied)  
    }  
}
```

Что изменилось за последние релизы — это не «мелкие детали», это меняет UX-флоу

- **Android 11:** one-time permissions (пользователь может выдать доступ «только сейчас»), auto-reset разрешений для приложений, которыми давно не пользовались.
- **Android 13:** отдельный runtime-permission `POST_NOTIFICATIONS` — до этого уведомления показывались без запроса разрешения вообще, теперь без явного `requestPermissions` уведомления просто не появятся. Плюс — гранулярные media-разрешения `READ_MEDIA_IMAGES`/`READ_MEDIA_VIDEO`/`READ_MEDIA_AUDIO` вместо одного общего `READ_EXTERNAL_STORAGE`.
- **Photo Picker** (`ActivityResultContracts.PickVisualMedia`) — системный выбор фото/видео вообще без необходимости в storage-разрешении, если приложению не нужен постоянный доступ ко всей галерее, а нужен именно выбор конкретных файлов пользователем.
- **Android 16:** автоматический отзыв (auto-expiring) разрешений у приложений, которыми не пользовались месяцами — расширение идеи auto-reset из Android 11 на большее число сценариев.

UX-обязанность, а не только API-вызов

Официальная рекомендация — всегда предусматривать путь работы приложения при отказе в разрешении (`rationale UI` перед повторным запросом через `shouldShowRequestPermissionRationale()`, graceful degradation функциональности) — «просто запросить ещё раз в цикле, пока не разрешит» не только противоречит гайдлайнам Google Play, но и не работает технически: после двух отказов система перестаёт показывать системный диалог вообще (`NEVER_ASK_AGAIN`), и единственный путь — сопроводить объяснением, ведущим в настройки приложения.

13. Activity Result API вместо `startActivityResult`

`startActivityResult()`/`onActivityResult()` deprecated — весь механизм завязан на числовые `requestCode`, которые легко перепутать между разными частями экрана, и колбэк приходит в `Activity`/`Fragment` напрямую, разрывая инкапсуляцию (любой код, вызвавший `startActivityResult`, обязан ещё и знать про `onActivityResult` того же класса).

kotlin

```
private val pickImageLauncher = registerForActivityResult(  
    ActivityResultContracts.GetContent()  
) { uri: Uri? ->  
    uri?.let { viewModel.acceptAction(EditorAction.ImageSelected(it)) }  
}  
  
// ВЫЗОВ:  
pickImageLauncher.launch("image/*")
```

`registerForActivityResult` регистрируется в поле класса (не в момент клика!) — обязательно до `onStart()` жизненного цикла компонента, иначе бросается исключение; это единственная практическая сложность миграции с legacy-подхода, где регистрация могла происходить в любой момент.

14. Manifest и Gradle: где что настраивается

Конфигурация приложения — в Gradle DSL, не в Manifest

`versionCode`, `versionName`, `minSdk`, `targetSdk`, `applicationId`, `namespace` задаются в module-level (`build.gradle.kts`), а не через атрибуты `AndroidManifest.xml` — Android Gradle Plugin инжектит эти значения в финальный манифест на этапе сборки:

```
kotlin

// build.gradle.kts (:app)
android {
    namespace = "com.example.app"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.example.app"
        minSdk = 26
        targetSdk = 36 // API 36 — обязательный минимум для Google Play с 31.
        versionCode = 142
        versionName = "4.12.0"
    }
}
```

Version Catalogs (`libs.versions.toml`) — централизованное управление версиями зависимостей вместо разбросанных строк по модулям, с типобезопасными ссылками (`libs.retrofit`) вместо голых строк координат.

Что остаётся в `AndroidManifest.xml`

Декларация компонентов (`<activity>`, `<service>`, `<receiver>`, `<provider>`), permissions (`<uses-permission>`), intent-filters, `<application>`-атрибуты уровня приложения (theme, application class). Ничего из версионирования и SDK-таргетов здесь больше не место — если увидите `<uses-sdk android:minSdkVersion=...>` в объяснении «как настроить проект» — это устаревший материал.

`android:exported` — обязателен явно с API 31

Начиная с Android 12 (API 31) любой компонент с `intent-filter` обязан явно указывать `android:exported="true"` или `"false"` — сборка просто не пройдёт без этого атрибута. Это не формальность: неявно `exported`-компонент, доступный сторонним приложениям без осознанного решения — классический вектор атаки (`intent redirection`, произвольный запуск активности/сервиса извне). Правило по умолчанию: `exported="false"`, если только компонент не должен принимать `intent`'ы явно от других приложений (например, `deeplink`-обработчик).

15. Логирование: Timber

`android.util.Log` не используется — только `Timber`, с обязательным ручным `TAG`, а не `this::class.java.simpleName`:

kotlin

```
class BenefitsActor @Inject constructor(
    private val repository: BenefitsRepository,
) : Actor<BenefitsState, BenefitsAction, BenefitsEffect> {

    private companion object {
        const val TAG = "BenefitsActor" // <-- жёстко захардкожено, не reflect
    }

    override fun invoke(action: BenefitsAction, state: BenefitsState) = flow {
        Timber.tag(TAG).d("mylog 001: Starting benefits calculation, userId=$userId")
        // ...
    }.catch { e ->
        Timber.tag(TAG).e("mylog 002: Benefits fetch failed: ${e.message}")
        emit(BenefitsEffect.ErrorOccurred(e.message.orEmpty()))
    }
}
```

Почему не `this::class.java.simpleName`: при включённом R8/ProGuard обфускация переименовывает классы, и `simpleName` в релизной сборке возвращает бессмысленный короткий идентификатор (`a`, `b`, `Ac3`) вместо реального имени — теги логов становятся бесполезны именно там, где логи нужнее всего (продакшен-краши). Хардкод строки — единственный надёжный вариант.

Нумерация `mylog NNN`: уникальный номер на каждый лог-вызов внутри класса — позволяет мгновенно найти конкретное место в коде по номеру из Logcat/краш-репорта без текстового поиска по сообщению (которое могло измениться). В новом классе — начинать с 001; в существующем — продолжать с последнего использованного номера, не переиспользуя чужие.

16. Тестирование: JUnit5 + MockK + Flow/Compose

Что тестировать в MVI — в первую очередь Reducer

Reducer — чистая функция, поэтому это самые дешёвые и самые ценные тесты в проекте: без моков, без корутин, просто `assertEquals(expectedState, reducer(inputState, effect))`. Если в проекте есть баг с некорректным состоянием экрана — в 90% случаев это баг в Reducer, и покрытие именно этого компонента даёт максимальную защиту на минимум усилий.

kotlin

```
class ProductListReducerTest {
    private val reducer = ProductListReducer()

    @Test
    fun `DataLoaded effect moves state from Loading to Content`() {
        val result = reducer.invoke(
            state = ProductListState.Loading,
            effect = ProductListEffect.DataLoaded(listOf(fakeProduct)),
        )
        assertEquals(ProductListState.Content(listOf(fakeProduct)), result)
    }
}
```

Actor — мокаем зависимости, тестируем через Flow

kotlin

```
class ProductListActorTest {
    private val repository: ProductRepository = mockk()
    private val actor = ProductListActor(repository)

    @Test
    fun `ScreenOpened emits Loading then DataLoaded on success`() = runTest {
        coEvery { repository.fetchProducts() } returns listOf(fakeProduct)

        actor.invoke(ProductListAction.ScreenOpened, ProductListState.Loading)
        assertEquals(ProductListEffect.Loading, awaitItem())
        assertEquals(ProductListEffect.DataLoaded(listOf(fakeProduct)), awaitItem())
        awaitComplete()
    }
}
```

(`.test { }`) — из Turbine; `runTest` + `coEvery`/`mockk()` — стандартная связка JUnit5 + MockK + coroutines-test.)

MockK-специфика, которую путают с Mockito

`mockk(relaxed = true)` — заглушки для всех методов по умолчанию (в отличие от строгого мока, где незамоканный вызов роняет тест) — удобно для больших интерфейсов, где важны 1-2 метода, но нужно осторожно: `relaxed`-мок может маскировать забытый `coEvery` и тест пройдёт «зелёным» на неправильной логике. `every` vs `coEvery` — вторая обязательна для `suspend`-функций, обычная `every` на `suspend`-методе не скомпилируется/не сработает как ожидается.

Compose UI-тесты

`createComposeRule()` + `onNodeWithText(...)`/`onNodeWithTag(...)` — тестируют дерево семантики (accessibility tree), а не пиксели. Для MVI-экранов практичнее тестировать не весь `ProductListScreen`, а: (1) Reducer/Actor юнит-тестами (дёшево, быстро), (2) один-два UI-теста на критичный happy-path экрана целиком (дорого, медленно, но ловит интеграционные проблемы разметки/state-биндинга, которые юнит-тесты не видят).

17. Производительность и сборка

R8 — не «просто обфускация»

R8 одновременно делает три вещи: сжатие (shrinking — удаление неиспользуемого кода), обфускацию (переименование классов/полей) и оптимизацию байткода (inline, dead code elimination). Каждая ломает что-то своё без правильного `proguard-rules.pro`: shrinking удаляет классы, доступные только через reflection (Gson/Retrofit-модели без `@Keep`), обфускация ломает `simpleName`-based логику (см. раздел про Timber выше) и сериализацию по имени поля.

Baseline Profiles

Список «горячих путей» кода (какие методы/классы вызываются при старте приложения и в первых секундах ключевых экранов), который ART использует для АОТ-компиляции этих путей заранее, а не JIT-компиляции по требованию — сокращает время холодного старта и первый jank на критичном экране. Генерируются через `Macrobenchmark`-тесты (`BaselineProfileRule`), должны обновляться при значимых изменениях в стартовом пути приложения, а не генерироваться один раз и забываться.

Compose Stability annotations — см. раздел 4

Отдельно стоит помнить: `ImmutableCollections` (`kotlinx.collections.immutable`) или собственные `@Immutable`-враппер-классы для списков/мап в параметрах `composable` — конкретный, измеримый рычаг производительности, а не теоретическая рекомендация; `Layout Inspector/Compose Compiler Metrics` (`--include-source-information` + отчёт компилятора) показывают, какие `composable` реально помечены `skippable/restartable`, и это первое, что стоит проверить при жалобах на «тормозит список».

Модуляризация: `api` vs `implementation`

`implementation` — зависимость доступна только внутри модуля, не протекает транзитивно потребителям этого модуля (пересборка при изменении такой зависимости не требует пересборки модулей выше по графу). `api` — зависимость становится частью публичного API модуля и видна всем, кто зависит от него транзитивно. Правило по умолчанию: `implementation`, пока модуль реально не экспортирует типы из зависимости в своём публичном API — злоупотребление `api` — самая частая причина того, что модуляризация не даёт ожидаемого ускорения инкрементальной сборки, потому что граф пересборки становится плотнее, чем должен быть.

18. Шпаргалка: легаси → актуальный стек

Легаси	Актуально	Когда встретите легаси
XML Views / <code>findViewById</code>	Jetpack Compose	точно оборачивайте в <code>ComposeView</code> вместо переписывания всего экрана «заодно»
RxJava (<code>Observable</code> , <code>Subject</code>)	Coroutines + Flow	см. таблицу миграции в разделе 3 — главная ловушка: hot-по- умолчанию поведение <code>Subject</code>
<code>AsyncTask</code>	Coroutines (<code>viewModelScope</code>) / <code>WorkManager</code>	см. раздел 11 — критерий выбора: должна ли задача пережить смерть процесса
<code>startActivityForResult</code> / <code>onActivityResult</code>	<code>registerForActivityResult</code>	см. раздел 13
<code>SharedPreferences</code>	<code>DataStore</code>	синхронный/ блокирующий API → асинхронный на <code>Flow</code>
MVP (<code>Presenter</code> + <code>View</code> - интерфейс), Моху	MVI (<code>Actor</code> / <code>Reducer</code> / <code>NewsPublisher</code>)	см. раздел 6

<code><uses-sdk></code> , <code>versionCode</code> / <code>versionName</code> в Manifest	Gradle DSL (<code>build.gradle.kts</code>)	см. раздел 14
Permissions «выдаются при установке» (верно только для <code>normal</code>)	runtime-запрос для <code>dangerous</code> permissions	см. раздел 12
Service как универсальный background-инструмент	<code>WorkManager</code> для персистентной работы, Foreground Service только с notification + type	см. раздел 11

Собрано с прицелом на собеседования и самопроверку по актуальному (2026) продакшен-стеку: Jetpack Compose, Coroutines/Flow, MVI, Dagger 2 (ручной DI), кастомная Fragment-навигация, Room/DataStore, Retrofit/OkHttp, Timber, JUnit5/MockK.